

A Project Report

on

Complainify

Submitted in partial fulfillment of the requirement of

Project-VI (BIT356CO)

Bachelor in Information Technology

Submitted To



Purbanchal University

Biratnagar, Nepal

Submitted By

Mridul Prakash Shah (360818)

Om Prakash Basnet (360820)

Samir Poudel (360831)

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

October, 2026

A Project Report

on

Complainify

Submitted in partial fulfillment of the requirement of

Project-VI (BIT356CO)

Bachelor in Information Technology

Submitted To

Purbanchal University

Biratnagar, Nepal

Submitted By

Mridul Prakash Shah (360818)

Om Prakash Basnet (360820)

Samir Poudel (360831)

Project Supervisor

Er. Maryada Karki

Mr. Santosh Shah

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

October, 2026

Abstract

Educational institutions often face difficulties in managing student complaints because traditional methods such as paper forms, emails, or manual records lead to delays, poor organization, and a lack of transparency in the complaint resolution process. To address these issues, this project proposes Complainify, an AI-Based Student Complaint Analysis and Decision Support System that provides a centralized and user-friendly platform for handling student grievances. Through this system, students can submit complaints related to academics, examinations, administration, infrastructure, or faculty issues at any time and from any location. The system uses Artificial Intelligence techniques, including sentiment analysis and automatic text classification, to analyze the complaint content, identify its category, and determine its urgency level. Based on the analysis, complaints are automatically assigned priorities such as High, Medium, or Low and forwarded to the appropriate authorities for faster action. The system also includes an administrative dashboard that provides real-time complaint statistics, tracking facilities, filtering options, and report generation features to support effective decision-making. Additionally, an integrated email notification service keeps students updated about the status of their complaints and alerts administrators when high-priority issues are reported. Developed using the Iterative Waterfall Software Development Life Cycle model and implemented with technologies such as HTML, CSS, JavaScript, Python Flask, MySQL, and machine learning libraries, the proposed system aims to improve efficiency, enhance transparency, and ensure quicker and more effective resolution of student complaints in educational institutions.

Acknowledgement

We'd like to express our heartfelt gratitude to everyone who has encouraged us to work on this project. First and foremost, we'd like to express our gratitude to the entire team of Kantipur City College (KCC) for this chance, especially to the Principal of Kantipur City College, who assisted us in furthering our knowledge in this field.

And we'd like to express our special gratitude to our supervisor, Er. Maryada Karki and Mr. Santosh Shah who has consistently encouraged, inspired and provided us with a wealth of information that has been beneficial. His advice was helpful in completing this assignment. We could not have asked for a better supervisor, counselor, or mentor. This project would not have been possible without the assistance of each member, as well as the entire class, who offered suggestions, shared their experiences, and provided advice throughout the project. We are grateful for this. Finally, we'd like to express our gratitude to our friends and colleagues who supported us directly and indirectly throughout this project.

Declaration

We declare that this project report titled “Complainify” submitted in partial fulfillment of the Bachelor of Information Technology is a record of original work carried out by us under the supervision of Er. Maryada Karki and Mr. Santosh Shah and has not formed the basis for the award of any other degree or Diploma, in this or any other institution or University. In keeping with the ethical practice of reporting scientific information, due acknowledgments have been made wherever the findings of others have been cited.

Mridul Prakash Shah (360818)

Om Prakash Basnet (360820)

Samir Poudel (360831)

Supervisor's Approval

This is to certify that the project entitled “Complainify” undertaken and demonstrated by Mridul Prakash Shah, Om Prakash Basnet and Samir Poudel has been successfully completed under my supervision as a partial fulfillment of the requirements of the degree of IT 6th semester under Purbanchal University. I, henceforth, approve this project to be awarded the certificate by the concerned authority. During supervision, I found students hardworking, skilled and ready to undertake any professional work related to this field in future.

Mr. Santosh Shah
Project Supervisor
Department of IT
Date:

Er. Maryada Karki
Project Supervisor
Department of IT
Date:

Certificate From Department

Following the Supervisor's Approval and Examiner's Acceptance, the project entitled "Complainify" submitted by Mridul Prakash Shah, Om Prakash Basnet and Samir Poudel as a partial fulfillment of the requirements for the degree of BIT 6th Semester under Purbanchal University, has been officially awarded by this certificate. I wish the students all the best for their future endeavors.

Mr. Rubim Shrestha

Project Coordinator

Department of IT

Date:

Mr. Saroj Pandey

Head of Department

Department of IT

Date:

Table of Contents

Abstract	i
Acknowledgement	ii
Declaration	iii
Supervisor's Approval	iv
Certificate From Department	v
List of Figures	ix
List of Tables.....	x
List of Abbreviations.....	xi
1 Introduction.....	1
1.1 Overview:.....	1
1.2 Problem Statement:	1
1.3 Objectives:	2
1.4 Features:	2
1.5 Significance:	4
1.6 Scope and Limitations.....	4
1.7 Documentation Organization	5
2 Literature Review.....	6
2.1 Smart Complaint Management System with AI-Powered Powered Prioritization and Escalation	6
2.1.1 Working Principle	6
2.1.2 Advantages:.....	6
2.1.3 Limitations:	7
2.2 Machine Learning-Based Smart Students' Complaint System	7
2.2.1 Working Principle	7
2.2.2 Advantages:.....	7
2.2.3 Limitations:	7
2.3.1 Working Principle:	8

2.3.2 Advantages:.....	8
2.3.3 Limitations:.....	8
3 Methodology.....	9
3.1 System Development Cycle.....	9
3.2 Tools and Technologies Used.....	10
3.2.1 Programming Languages and Web Technologies	10
3.2.2 Database Management System	11
3.2.3 Python Libraries.....	11
3.2.4 Development Tools	11
3.2.5 Dataset Collection & Preprocessing	11
3.2.5 Model Training & Evaluation	12
3.3 Assignment of Roles and Responsibilities.....	12
4 System Design and Analysis.....	14
4.1 Requirements Analysis.....	14
4.1.1 Requirement Gathering.....	14
4.1.2 Functional Requirements	16
4.2 Feasibility Study	17
4.2.1 Technical Feasibility	17
4.2.2 Operational Feasibility.....	17
4.2.3 Economic Feasibility	18
4.2.4 Schedule Feasibility	21
5 System Design	23
5.1 System Architecture	23
5.1.1 Architectural Components	23
5.2 Procedure Oriented Diagram	25
5.2.1 Context Diagram (DFD Level 0)	25

5.2.2 Data Flow Diagram (Level -1).....	26
5.2.3 Use Case Diagram.....	27
5.3 Database Design.....	28
5.3.1 E-R Diagram	28
6 System Development and Implementation	29
6.1 Programming Platforms	29
6.2 Hardware and Software Requirements	29
6.2.1 Hardware Requirements.....	29
6.2.2 Software Requirements	30
7 Testing and Debugging	31
7.1 Testing Approach	31
7.2 Testing Tools	31
7.3 Test Cases.....	31
7.3.1 Login Module.....	32
7.3.2 Sign Up Module.....	33
7.3.3 Forgot Password / Reset Password Module	34
7.3.4 Complaint Submission Module.....	35
7.3.5 AI Classification Module (Naive Bayes)	36
7.3.6 Sentiment Analysis Module	36
7.3.7 Complaint Status Tracking Module	37
7.3.8 Admin Dashboard and Filtering Module	38
7.3.9 Email Notification Module	38
7.3.10 Report Generation Module	39
8 Conclusion	40
References.....	41

List of Figures

Figure 3.1: Iterative Waterfall Model	10
Figure 4.1: Gantt Chart	22
Figure 5.1: System Architecture	23
Figure 5.2: DFD Level 0	25
Figure 5.3: DFD Level 1	26
Figure 5.4: Use Case Diagram	27
Figure 5.5: ER Diagram	28

List of Tables

Table 3.1: Evaluation Metrics	12
Table 3.2: Assignment of Roles and Responsibilities	12
Table 4.1: Requirement Analysis	14
Table 4.2: Functional Requirements	16
Table 4.3: Initial Development Cost	18
Table 4.4: Net Cash Flow Analysis	19
Table 4.5: Calculation of NPV	20
Table 4.6: Summary of Financial Metrics	20
Table 5.1: System Architecture Components	23
Table 7.1: Testing Tools	31
Table 7.2: Login module	32
Table 7.3: SignUp Module	33
Table 7.4: Forgot Password / Reset Password Module	34
Table 7.5: Complain Submission Module	35
Table 7.6: AI Classification Module	36
Table 7.7: Sentimental Analysis Module	36
Table 7.8: Complaint Status Tracking Module	37
Table 7.9: Admin Dashboard and Filtering Module	38
Table 7.10: Email Notification Module	38
Table 7.11: Report Generation Module	39

List of Abbreviations

AI	:	Artificial Intelligence
API	:	Application Programming Interface
CPU	:	Central Processing Unit
CRUD	:	Create, Read, Update, Delete
CSS	:	Cascading Style Sheets
DFD	:	Data Flow Diagram
ER	:	Entity Relationship
HTML	:	HyperText Markup Language
HTTP	:	HyperText Transfer Protocol
IDE	:	Integrated Development Environment
IRR	:	Internal Rate of Return
JS	:	JavaScript
ML	:	Machine Learning
MNB	:	Multinomial Naive Bayes
MySQL	:	My Structured Query Language
NCF	:	Net Cash Flow
NPR	:	Nepalese Rupee
NPV	:	Net Present Value
PDF	:	Portable Document Format
PHP	:	Hypertext Preprocessor
RAM	:	Random Access Memory
SDLC	:	Software Development Life Cycle
SMTP	:	Simple Mail Transfer Protocol
SQL	:	Structured Query Language
TF-IDF	:	Term Frequency–Inverse Document Frequency
UI	:	User Interface
URL	:	Uniform Resource Locator

USB : Universal Serial Bus
XAMPP : Cross-Platform Apache MySQL PHP Perl

1 Introduction

1.1 Overview:

Complainify is an AI-Based Student Complaint Analysis and Decision Support System developed to automate the management of student complaints in educational institutions. Traditional complaint handling methods, such as paper forms and manual records, often lead to delayed responses, improper categorization, and a lack of transparency. To overcome these issues, the proposed system provides a centralized web-based platform where students can submit and track complaints efficiently. The system applies concepts from Artificial Intelligence and Data Mining, text classification, and sentiment analysis. When a complaint is submitted, the text is preprocessed and classified into categories such as Academic, Examination, Infrastructure, or Administration using the Naïve Bayes Classification Algorithm, which is implemented from scratch in Python due to its simplicity and effectiveness in text classification tasks.

The system also performs sentiment analysis and assigns a priority level (High, Medium, or Low) to support administrators in making timely decisions which are developed from scratch using the collected data for training and testing, without using pre-trained machine learning models. Also, it supports voice-based complaint input which supports only English language. Data mining techniques are used to identify complaint patterns and generate reports, dashboards, and graphical statistics that help authorities understand recurring issues and make data-driven decisions. Additionally, an email notification system is integrated to send complaint acknowledgements, status updates, and alerts for high-priority complaints to both students and administrators. The system is implemented using HTML, CSS, JavaScript, Python Flask, and MySQL, along with supporting libraries such as Pandas.

1.2 Problem Statement:

Most educational institutions still manage student complaints manually through paper forms, emails, or unstructured communication channels. These traditional approaches suffer from several limitations:

- Complaints are not properly categorized.

- Urgent issues are often delayed due to the absence of prioritization mechanisms.
- Administrators lack analytical insights regarding complaint trends.
- Students do not receive timely updates regarding complaint status.
- There is no centralized platform to monitor and manage complaints efficiently.

Therefore, there is a need for an intelligent, automated, and centralized complaint management system that can analyze complaints, determine their priority, notify concerned authorities through email, and assist administrators in making data-driven decisions.

1.3 Objectives:

The main objective of the project is:

- To develop a web-based AI-powered system for automated student complaint analysis and prioritization

1.4 Features:

Complainify is designed with several features to ensure usability, efficiency, and reliability:

1. **User Registration and Authentication:** Secure multi-role registration and login system for Students and Administrators, with email-based password reset and account verification features.
2. **Complaint Submission System:** Students can submit complaints related to academics, examinations, infrastructure, administration, or faculty issues by providing complaint details and supporting information through an online portal.
3. **Anonymous Complaint Submission:** Students can choose to submit complaints anonymously, ensuring privacy and encouraging the reporting of sensitive issues without fear of identification.
4. **AI-Based Complaint Classification:** The system automatically analyzes complaint text using Artificial Intelligence and Natural Language Processing techniques to classify complaints into appropriate categories and departments.

5. **Voice Complaint Input:** Allows users to enter complaints using voice through the Web Speech API which supports only English language.
6. **Sentiment Analysis:** The system performs sentiment analysis on complaint content and identifies whether the complaint expresses positive, neutral, or negative sentiments to better understand the severity of issues.
7. **Automatic Priority Assignment:** Based on complaint content, sentiment, and predefined rules, the system automatically assigns priority levels such as High, Medium, or Low to assist administrators in handling urgent complaints efficiently.
8. **Complaint Status Tracking:** Students can monitor the status of their complaints, including Submitted, Under Review, In Progress, Resolved, or Rejected, ensuring transparency throughout the complaint resolution process.
9. **Search and Filter Functionality:** Administrators can search and filter complaints based on category, department, priority, status, date, or student information for efficient complaint management.
10. **Dashboard and Analytics:** Role-specific dashboards provide complaint statistics, category-wise distributions, monthly trends, priority analysis, and graphical reports to support data-driven decision-making.
11. **Email Notification System:** Automated email notifications are sent for complaint submission confirmations, status updates, high-priority complaint alerts, and resolution notifications.
12. **PDF and Excel Report Generation:** Administrators can generate and export complaint reports in PDF and Excel formats containing complaint summaries, statistics, and analytical insights.
13. **Decision Support System:** The system utilizes Data Mining and AI techniques to identify complaint patterns, frequently occurring issues, and departmental performance trends, assisting management in making informed decisions.
14. **Complaint History and Audit Trail:** The system maintains a complete history of complaint activities, including submissions, status changes, and administrative actions for accountability and future reference.

15. Responsive User Interface: A clean, modern, and user-friendly interface is provided for both students and administrators, ensuring accessibility across desktop and mobile devices with minimal technical knowledge.

1.5 Significance:

Complainify is significant because it provides an intelligent and centralized platform for managing student grievances efficiently by automating complaint submission, classification, prioritization, and tracking. The system improves transparency and communication between students and administrators through real-time status updates and email notifications while reducing manual effort and delays in complaint resolution. By integrating Artificial Intelligence and Data Mining techniques such as text classification, sentiment analysis, and pattern discovery, the system helps institutions identify recurring issues, analyze complaint trends, and make informed decisions through analytical dashboards and reports. The project demonstrates the practical application of AI and Data Mining concepts in solving real-world problems and contributes to improving student satisfaction and institutional performance.

1.6 Scope and Limitations

Scope

The scope of Complainify is to develop a web-based intelligent complaint management system that allows students to submit and track complaints while enabling administrators to manage and analyze them efficiently. The system uses Artificial Intelligence and Data Mining techniques such as Natural Language Processing, Naïve Bayes Classification, and sentiment analysis to categorize complaints, assign priorities, generate reports, and provide email notifications, thereby improving transparency and supporting data-driven decision-making in educational institutions.

Limitations

While developing the project, some features could not be implemented due to time and resource limitations:

- Integration with external university management systems is not implemented.

- Multi-language support is currently unavailable.
- Advanced AI-based recommendation and predictive complaint resolution features are not included.
- Real-time mobile push notification functionality is not implemented, and the system currently relies on email notifications.
- The accuracy of complaint classification depends on the quality and quantity of training data available.

1.7 Documentation Organization

Chapter 1 introduces our project by explaining its purpose, importance, objectives, features, scope, and limitations, providing an overview of the system. Chapter 2 reviews existing medication reminder systems, compares their strengths and limitations, and identifies gaps that justify the development of the proposed system. Chapter 3 describes the methodology used to develop the project, including the Software Development Life Cycle, tools, technologies, programming languages, database systems, data collection and preprocessing process and team roles. Chapter 4 focuses on system design and requirement analysis, including functional requirements, feasibility study, and system architecture. Chapter 5 presents system diagrams such as Data Flow Diagrams, Entity Relationship Diagrams, Use Case Diagrams, and Gantt charts to illustrate system workflow and project planning. Chapter 6 explains system development and implementation details, including hardware and software requirements. Chapter 7 discusses system testing and debugging, presenting test cases and results. Finally, the conclusion summarizes project achievements and suggests future improvements.

2 Literature Review

Efficient complaint management is important in educational institutions to ensure that students' concerns are addressed effectively and within an appropriate timeframe. Traditional complaint handling approaches are mostly manual and often suffer from delayed responses, lack of categorization, and difficulties in identifying urgent issues. To overcome these problems, several intelligent complaint management and feedback analysis systems have been proposed. These systems employ machine learning, Natural Language Processing, sentiment analysis, and automated prioritization to improve complaint handling and support decision making. Since the proposed system is developed as a web-based application with AI-assisted analysis, reviewing existing systems helps identify their features, strengths, and limitations. Most existing systems focus on complaint classification or sentiment analysis, while the proposed system combines sentiment analysis, automatic categorization, priority assignment, and dashboard-based reporting

2.1 Smart Complaint Management System with AI-Powered Powered Prioritization and Escalation

The Smart Complaint Management System with AI-Powered Prioritization and Escalation is an AI-based complaint handling system designed to improve the efficiency of complaint management through Natural Language Processing and automated prioritization. The system classifies complaints and determines their urgency to ensure timely responses and effective resolution (Deo et al., 2025).

2.1.1 Working Principle

The system provides a web interface through which users can submit complaints. After submission, Natural Language Processing techniques are used to analyze the complaint text and classify it into appropriate categories. Sentiment analysis and AI-based priority assignment are used to determine the severity of complaints and automatically escalate urgent cases to the concerned authorities (Deo et al., 2025).

2.1.2 Advantages:

- Reduces manual effort in complaint categorization.
- Provides automatic priority assignment.
- Supports dashboard-based monitoring and reporting.

- Enhances transparency in complaint handling

2.1.3 Limitations:

- Performance depends on the quality of training data.
- Incorrect sentiment analysis may affect priority assignment.
- Requires periodic model updates.
- Limited support for multilingual complaints

2.2 Machine Learning-Based Smart Students' Complaint System

The Machine Learning-Based Smart Students' Complaint System was developed to automate the handling of academic complaints and improve the efficiency of complaint resolution in educational institutions. The system employs machine learning and deep learning techniques to classify complaints according to departments and identify the corresponding issue categories (Javed et al., 2025).

2.2.1 Working Principle

Students submit complaints through the system, and the complaint data are processed using Natural Language Processing techniques. Various algorithms including Support Vector Machine, Random Forest, Decision Tree, Convolutional Neural Network, and BERT are applied for department classification and aspect identification. The classified complaints are then forwarded to the appropriate department for further action (Javed et al., 2025).

2.2.2 Advantages:

- Automates complaint classification.
- Supports multiple machine learning algorithms.
- Improves efficiency of complaint resolution.
- Reduces dependency on manual processing.
- Provides department-wise categorization.

2.2.3 Limitations:

- Accuracy depends on dataset quality.
- Limited ability to detect implicit meanings in complaints.
- Requires a large amount of training data

2.3 AI-Driven Real-Time Feedback System for Enhanced Student Support

The AI-Driven Real-Time Feedback System for Enhanced Student Support is a machine learning-based framework that integrates sentiment analysis and predictive algorithms to provide personalized support and improve student engagement. The system analyzes student interactions and emotional states to generate context-sensitive feedback (Prakash et al., 2024).

2.3.1 Working Principle:

The system collects data from student interactions such as forum posts, assignment submissions, and feedback forms. Sentiment analysis is applied to identify emotional patterns, while machine learning algorithms such as Decision Trees, Support Vector Machines, Random Forest, and Deep Learning models are used to predict student engagement and performance. Based on the analysis, personalized recommendations and interventions are provided to support students academically and emotionally (Prakash et al., 2024).

2.3.2 Advantages:

- Provides real-time analysis of student feedback.
- Enhances student engagement and satisfaction.
- Supports personalized recommendations.
- Improves prediction accuracy using sentiment information.
- Enables data-driven decision making

2.3.3 Limitations:

- Deep learning models require high computational resources.

3 Methodology

3.1 System Development Cycle

For the development of Complainify: the Iterative Waterfall Model was selected as the software development methodology. The Iterative Waterfall Model is a structured and sequential approach that divides the development process into clearly defined phases. Each phase must be completed before moving to the next phase, ensuring systematic development, proper documentation, and effective project management.

The Iterative Waterfall Model was chosen for the following reasons:

1. Clear and Fixed Requirements:

The Student submission, complaint classification, sentiment analysis, priority assignment, dashboard generation, and email notifications. Since these requirements remain stable throughout development, the Iterative Waterfall Model is suitable for the project. Complainify system has well-defined requirements such as user registration, complaint. However, feedback received during project reviews and defense sessions can be incorporated to refine and improve the system in subsequent iterations.

2. Easy to Manage:

The project follows a step-by-step development approach where each phase has specific tasks and deliverables. This makes it easier to manage, monitor progress, and ensure that all modules are completed systematically, which is ideal for a student-level project.

3. Better Documentation:

This Model emphasizes documentation at every stage, including requirement specifications, system design diagrams, database design, implementation details, and testing reports. Proper documentation is essential for academic projects and future maintenance.

4. Suitable for Academic Projects:

This Model is simple, easy to understand, and suitable for small development teams. It helps students complete the project within a limited academic time frame while maintaining a structured development process.

5. Predictable Timeline:

Since development follows a sequential process, it is easier to estimate the time required for each phase and complete the project according to the academic schedule.

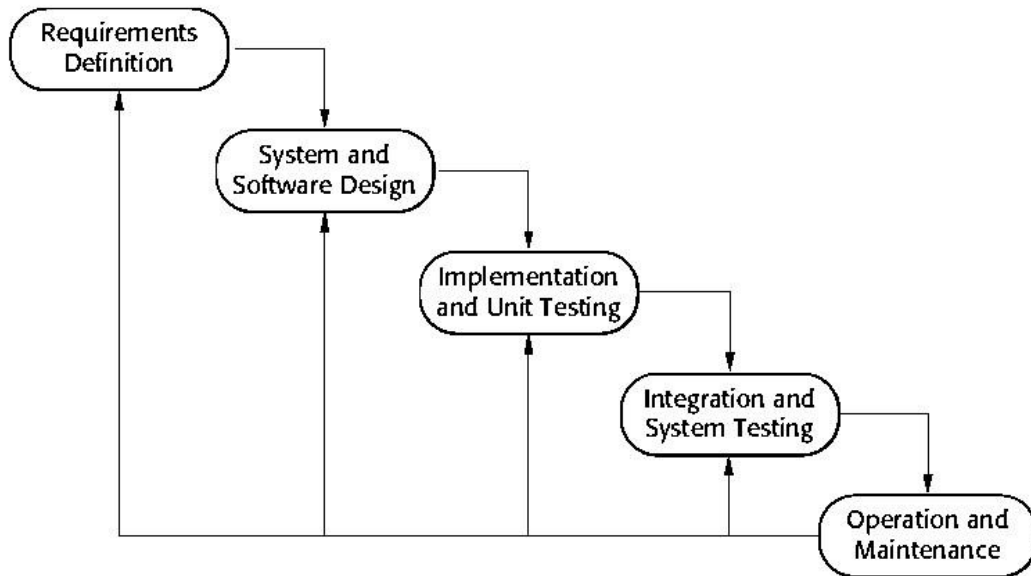


Fig 3. 1: Iterative Waterfall Model

3.2 Tools and Technologies Used

Complainify is developed using modern web technologies, Artificial Intelligence techniques, and a relational database management system to provide an efficient and intelligent complaint management platform. The system combines frontend and backend technologies with machine learning and natural language processing techniques to automate complaint classification, sentiment analysis, priority prediction, complaint tracking, and report generation.

3.2.1 Programming Languages and Web Technologies

The system is developed using HTML5, CSS3, and JavaScript for the frontend, while Python and the Flask framework are used for backend development. HTML structures the web pages, CSS designs the user interface, JavaScript adds interactivity, and Flask handles routing, business logic, AI integration, and communication with the MySQL database.

3.2.2 Database Management System

MySQL is used as the database management system to store student information, administrator accounts, complaint records, AI prediction results, complaint status, uploaded files, and notification history. The database is designed using relational tables to ensure data consistency, integrity, and efficient data retrieval.

3.2.3 Python Libraries

The system uses several Python libraries to support different functionalities. Flask is used for web development, Pandas and NumPy for data processing, PyMySQL for database connectivity, and Flask-Mail for sending automated email notifications.

3.2.4 Development Tools

The project is developed using Visual Studio Code as the primary code editor and Anaconda for Python environment management. MySQL is managed through XAMPP, while Git and GitHub are used for version control and project management. Web Speech API is used for voice input for the complaint description.

3.2.5 Dataset Collection & Preprocessing

- **Data Collection:** Complaint data was collected from public sources like Kaggle and other internet sources.
- **Category Dataset:** Around 9,000 complaint records were collected and adapted to student complaints.
- **Sentiment & Priority Dataset:** 12,000 records were prepared — 186 handwritten + 11,814 generated samples.
- **Cleaning:** Removed irrelevant, incomplete, duplicate, and very short records.
- **Encoding:** Data was normalized using UTF-8 and unnecessary characters were removed.
- **Label Encoding:**
 - **For sentimental analysis:** Positive = 0, Neutral = 1, Negative = 2
 - **For priority based:** Low = 0, Medium = 1, High = 2
 - **Category Encoding:** Academics = 0, Hostels = 1, IT Support = 2, Infrastructure = 3, Financial Services = 4, Administrative = 5,

Security = 6, Maintenance = 7, Transport = 8, Library = 9, Canteen = 10, and Other = 11.

- **Tokenization:** Lowercasing → noise removal → tokenization → stop word removal → short-word removal → stemming → bigrams.
- **Train-Test Split:** 80% training and 20% testing using stratified splitting.

3.2.5 Model Training & Evaluation

- **Algorithm:** Multinomial Naive Bayes, implemented from scratch
- **Models:** Category, Sentiment, and Priority classifiers.
- **Evaluation Metrics:** Accuracy, Precision, Recall, and F1-score.

Table 3.1: Evaluation Metrics

Model	Accuracy	Precision	Recall	F1-score
Category	96.81%	0.9691	0.9685	0.9687
Sentiment	95.12%	0.9254	0.9666	0.9406
Priority	98.12%	0.9811	0.9855	0.9833

3.3 Assignment of Roles and Responsibilities

Table 3.2: Assignment of Roles and Responsibilities

Group Member	Coding Responsibilities	Documentation Responsibilities
Mridul Prakash Shah	• Flask backend API • MySQL database design • Authentication & sessions • ML model integration • Priority assignment • Admin dashboard • Complaint management	• Feasibility Study • Database Design • Requirements Gathering • Methodology
Samir Poudel	• Landing page & Login/Signup • Forgot/Reset Password • Student Dashboard • Complaint form	• Introduction • System Architecture • Use Case Diagram • DFD Level 0 • Literature Review

	• Complaint tracking • Profile & Settings • Data collection & labeling • Naive Bayes training • Email notifications	
Om Prakash Basnet	• Dashboard JavaScript • Form validation • Reports (PDF/Excel) • Frontend–backend integration • Sentimental Analysis Training • Testing & debugging	• Functional Requirements • DFD Level 1 • Test Cases (Chapter 7)

4 System Design and Analysis

4.1 Requirements Analysis

Complainify was developed to address the growing need for an intelligent, automated, and centralized platform for managing student grievances in educational institutions. The requirements were gathered through analysis of existing manual and paper-based complaint-handling practices, discussions among the team regarding common bottlenecks faced by students and administrative staff, and a review of similar AI-based complaint and feedback management systems examined in the literature review. The system aims to bridge the gap between students and institutional administration by offering a centralized platform for complaint submission, AI-based classification and prioritization, real-time status tracking, and dashboard-based decision support for administrators.

4.1.1 Requirement Gathering

Requirements were gathered by analyzing traditional complaint-handling methods, communication gaps between students and administrators, and existing AI-based complaint systems. The key requirements include role-based access, complaint classification, sentiment analysis, priority assignment, real-time tracking, email notifications, and dashboard analytics.

Table 4.1: Requirement Gathering

Requirement ID	Description
R1	The system shall provide separate authentication mechanisms for Students and Administrators using email and password.
R2	Students shall be able to register, log in, manage their profile, and securely update their account information.
R3	Students shall be able to submit complaints by providing a title, description, category, and optional supporting attachments.
R4	The system shall automatically classify submitted complaints into appropriate categories using a Multinomial Naive Bayes model.
R5	The system shall perform sentiment analysis on complaint text using a custom rule-based approach.

R6	The system shall automatically predict the priority level of each complaint based on its content.
R7	Students shall be able to view the complete history and current status of their submitted complaints.
R8	Students shall receive email notifications when a complaint is submitted, updated, or resolved.
R9	Administrators shall be able to view, search, filter, and manage all submitted complaints
R10	Administrators shall be able to update complaint status, assign priorities, and provide responses to students.
R11	Administrators shall be able to manage student accounts, including activation, deactivation, and profile updates..
R12	The system shall provide dashboards displaying complaint statistics, trends, and category-wise reports.
R13	The system shall generate analytical reports based on complaint categories, priorities, and sentiments.
R14	The system shall store all complaint records, AI prediction results, and user information securely in a MySQL database.
R15	The system shall support secure file uploads for complaint evidence such as images and documents.
R16	The system shall implement role-based access control to ensure that users can only access authorized resources.
R17	The system shall validate all user inputs and prevent invalid or malicious data from being stored.
R18	The system shall maintain logs of complaint activities and administrative actions for auditing purposes.
R19	The system shall support voice-based complaint input using the Web Speech API.

4.1.2 Functional Requirements

The functional requirements define the specific behaviors and operations the system must perform to meet user needs and business objectives. Each functional requirement is mapped to one or more gathered requirements to ensure full coverage. Table 4.2 details all functional requirements for MedTracker.

Table 4.2: Functional Requirements

Function ID	Description	Cross Reference
F1	Authenticate Students and Administrators using secure login credentials.	R1
F2	Register student accounts and manage user profiles.	R2
F3	Submit complaints with title, description, category, and supporting attachments.	R3
F4	Automatically classify complaints into predefined categories using the AI model.	R4
F5	Perform sentiment analysis on submitted complaint text.	R5
F6	Predict the priority level of complaints using AI.	R6
F7	View complaint history and monitor complaint status.	R7
F8	Send email notifications for complaint submission and status updates.	R8
F9	View, search, and filter complaints using different criteria.	R9
F10	Update complaint status, assign priorities, and respond to complaints.	R10
F11	Manage student and administrator accounts	R11
F12	Display complaint statistics and dashboard summaries	R12
F13	Generate analytical reports based on complaint data.	R13
F14	Store and retrieve complaint records securely from the MySQL database.	R14
F15	Upload and manage supporting files for complaints.	R15

F16	Enforce role-based access control for secure system access	R16
F17	Validate user inputs and send automated email notifications.	R17
F18	Maintain activity logs and ensure secure handling of complaint information.	R18
F19	support voice-based complaint input using the Web Speech API.	R19

4.2 Feasibility Study

4.2.1 Technical Feasibility

Technical feasibility evaluates whether the available technologies, tools, and technical expertise are sufficient to develop the proposed system successfully.

Student Complainify is developed using widely adopted technologies including HTML, CSS, JavaScript, Python (Flask), MySQL, Pandas, NumPy and the Multinomial Naive Bayes algorithm. These technologies are open-source, well documented, and supported by large developer communities, making system development and maintenance practical.

The project can be developed using standard computing hardware without requiring specialized infrastructure or expensive software licenses. The development team possesses the necessary knowledge of web development, database management, and machine learning to implement the proposed solution. Therefore, the project is considered technically feasible.

4.2.2 Operational Feasibility

Operational feasibility evaluates whether the proposed system can be effectively used and accepted by its intended users. It focuses on the practicality of implementing the system within the organization and determining whether it satisfies user requirements while improving existing operations.

Student Complainify is designed with a simple, user-friendly, and responsive web interface that enables students and administrators to perform their tasks efficiently. Students can easily submit complaints, monitor complaint status, and receive timely

notifications, while administrators can manage complaints, monitor complaint trends, and generate reports through an intuitive dashboard.

The integration of Artificial Intelligence further reduces manual effort by automatically classifying complaints, predicting their priority, and analyzing sentiment, allowing administrators to respond more efficiently. Since the system requires only basic computer knowledge and web browser access, minimal user training is required. Therefore, the proposed system is considered operationally feasible.

4.2.3 Economic Feasibility

Complainify is economically feasible and demonstrates strong financial viability. The project requires an initial investment of NPR 175,000 to cover system development, AI model implementation, documentation, hardware, software setup, testing, training, and contingency costs. Annual operating expenses range from NPR 31,000 in Year 1 to NPR 40,000 by Year 5 for web hosting, maintenance, technical support, database management, and email services. The system provides measurable benefits by reducing manual complaint handling, improving administrative efficiency, enhancing transparency, and supporting AI-based complaint classification and sentiment analysis. These benefits are estimated to increase from NPR 120,000 in Year 1 to NPR 188,000 by Year 5, confirming the long-term financial sustainability of the proposed system.

Table 4.3: Initial Development Cost

Cost Component	Description	Estimated Cost (NPR)
Labor Cost (Development)	3 developers × 2 months × NPR 15,000	90,000
System Design & Documentation	Diagrams, reports, and testing documents	15,000
Hardware & Utilities	Laptop usage, hosting, internet, electricity	20,000

Email Service Setup and Voice Input API Setup	Flask-Mail SMTP configuration, Web Speech API and deployment	10,000
Training & Deployment	Administrator training, system deployment, and configuration	20,000
Contingency Cost	Unexpected or miscellaneous expenses	20,000
Total Initial Investment		175,000

Table 4.4: Net Cash Flow Analysis

Year	Annual Benefits (NPR)	Operating Costs (NPR)	Net Cash Flow (NPR)
1	120,000	31,000	89,000
2	135,000	33,000	102,000
3	150,000	35,000	115,000
4	165,000	38,000	127,000
5	188,000	40,000	148,000

Payback Period (PBP)

The Payback Period determines how quickly the initial investment is recovered.

Formula:

$PBP = \text{Year before recovery} + (\text{Remaining Investment} / \text{Next Year NCF})$

Cumulative Cash Flow:

- Year 1 = 89,000
- Year 2 = 191,000

Calculation:

$PBP = 1 + (175,000 - 89,000) / 102,000$

$PBP = 1.84 \text{ years} \approx \mathbf{1 \text{ year } 10 \text{ months}}$

The investment is recovered in less than 2 years, indicating low financial risk.

Net Present Value (NPV)

Formula:

$PV = NCF / (1 + r)^t$

$NPV = \Sigma PV - \text{Initial Investment}$

(Discount Rate = 12%)

Table 4.5: Calculation of NPV

Year	NCF (NPR)	Present Value (NPR)
1	89,000	79,464
2	102,000	81,309
3	115,000	81,847
4	127,000	80,710
5	148,000	83,998

NPV Calculation:

$$\text{NPV} = 79,464 + 81,309 + 81,847 + 80,710 + 83,998 - 175,000$$

$$\text{NPV} \approx \text{NPR } 240,559$$

Positive NPV confirms the project is financially viable.

Internal Rate of Return (IRR)

The IRR is the discount rate at which $\text{NPV} = 0$.

Using financial calculation methods:

$$\text{IRR} \approx 48.7\%$$

This is significantly higher than the required return (12–15%), indicating high profitability.

Table 4.6: Summary of Financial Metrics

Metric	Value
Payback Period	1 year 10 months
Net Present Value	NPR 240,559
Internal Rate of Return	48.7%

Commercial Value and Selling Price

From a commercial perspective, Complainify has strong market potential. Its key features include:

- AI-based complaint category prediction
- Sentiment analysis and priority prediction
- Centralized digital complaint management
- Email notification system

- Complaint tracking and analytical reports
- Administrative dashboard with complaint statistics

The system can be marketed to:

- Schools
- Colleges
- Universities
- Educational training institutes

Estimated Selling Price: NPR 150,000 per system

Conclusion

The economic analysis shows that Complainify is financially sound. It offers:

- Short payback period
- High NPV
- Strong IRR
- Low operational cost

Thus, the system ensures quick investment recovery, low financial risk, and long-term sustainability, making it a highly feasible and profitable solution.

4.2.4 Schedule Feasibility

The schedule feasibility analysis confirms that the project can be completed within a planned 15-week timeframe, spanning from June to September. The project begins with discussion and initiation (June W2), followed by requirement analysis (June W2–W4). Data selection is carried out during July W1, followed by data exploration and preprocessing (July W2). System design is performed from July W2 to W3, while model training takes place during July W3–W4. Frontend and backend development are conducted in parallel from July W4 to September W1. System testing is carried out from August W2 to September W2, followed by bug fixing from September W2 to W4. Documentation is maintained continuously throughout the project from June W2 to September W4. The logical sequence of activities, parallel development, and continuous documentation demonstrate that the project is schedule feasible and can be successfully completed within the proposed timeframe.

PROJECT TIMELINE (June – September)

Project Start Date: 13 June 2025 (Week 2 of June)

ACTIVITY	JUNE			JULY				AUGUST				SEPTEMBER			
	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
Project Start (Discussion)															
Requirement Analysis															
Data Selection															
Data Exploration and Preprocessing															
System Design															
Model Training															
Frontend Development															
Backend Development															
Testing															
Fixing Bugs															
Documentation															

Notes: W = Week | Total Duration: 15 Weeks

Fig 4.1: Gantt Chart

5 System Design

5.1 System Architecture

Complainify follows a multi-tier architecture consisting of the Presentation Layer, Business Logic Layer, Data Access Layer, Artificial Intelligence Layer, and External Integration Layer. This architecture improves modularity, maintainability, scalability, and security by separating different system functionalities into independent layers. Each layer performs a specific task and communicates with the adjacent layers through well-defined interfaces, making the system easier to develop, maintain, and extend with additional features in the future.

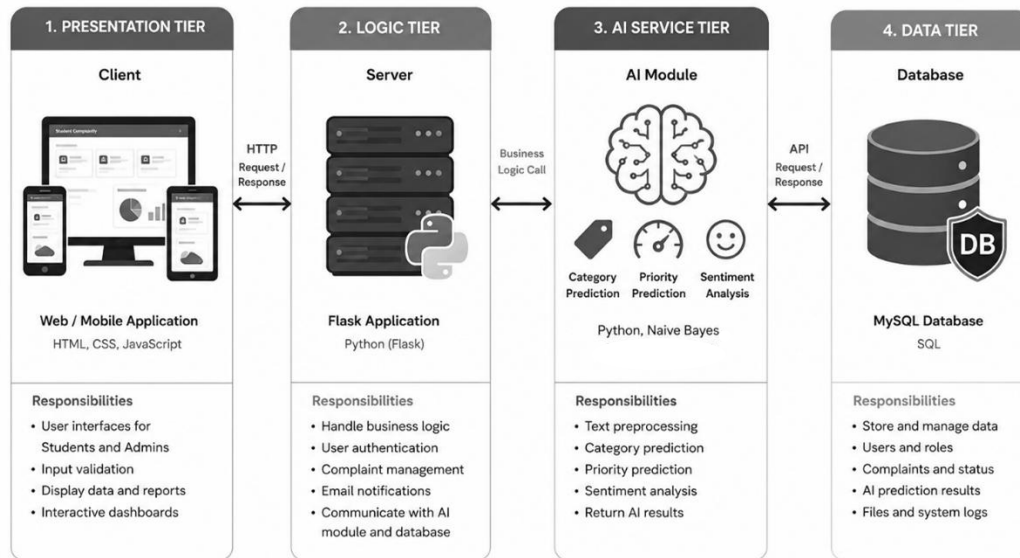


Fig 5.1: System Architecture

5.1.1 Architectural Components

Table 5.1: System Architecture Components

Layer	Technology	Description
Presentation Layer	HTML, CSS, JavaScript (Vanilla)	Provides interfaces for Students and Administrators, handles user input, and displays system information.
Business Logic Layer	Python (Flask)	Processes user requests, manages authentication, complaint handling,

		notifications, and business logic.
Artificial Intelligence Layer	Python, Multinomial Naive Bayes	Performs complaint classification, priority prediction, and sentiment analysis.
Data Access Layer	MySQL	Stores and manages user accounts, complaints, AI results, and other system data.
External Integration Layer	Flask-Mail (SMTP), Web Speech API.	Flask Mail Sends automated email notifications and supports external services such as file uploads and Web Speech API used for the voice-based complaint input in the system.

5.2 Procedure Oriented Diagram

5.2.1 Context Diagram (DFD Level 0)

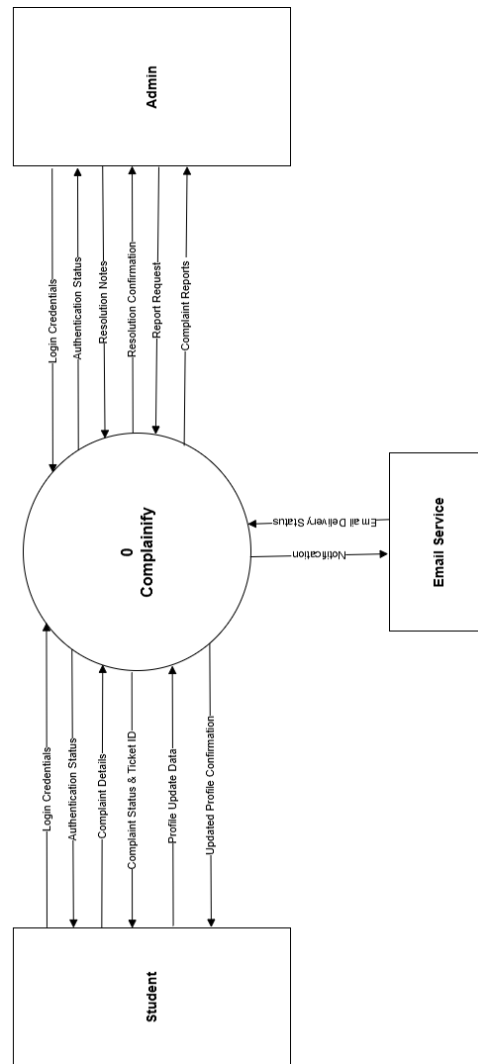


Fig 5.2: DFD Level 0

5.2.2 Data Flow Diagram (Level -1)

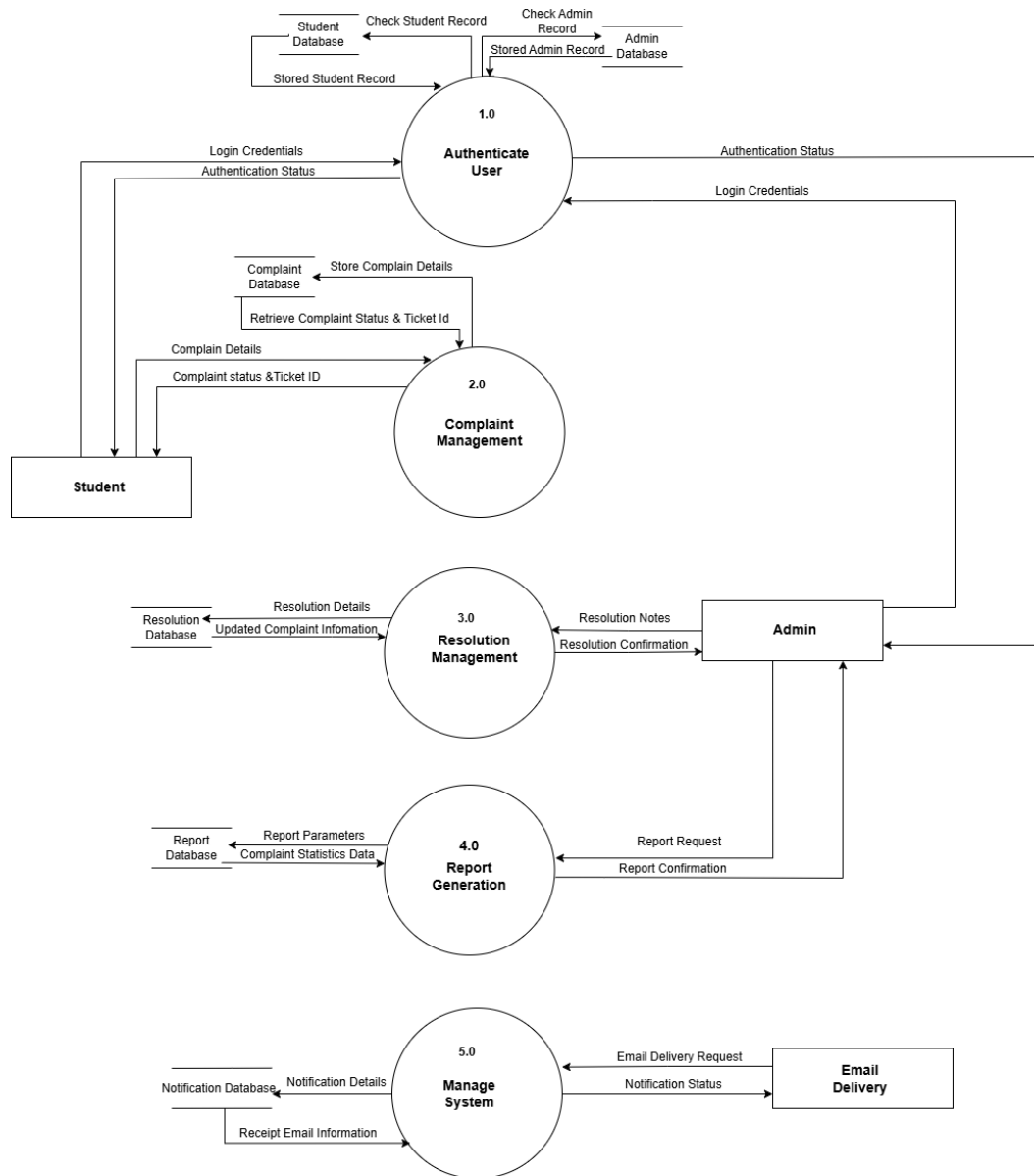


Fig 5. 3: DFD Level 1

5.2.3 Use Case Diagram

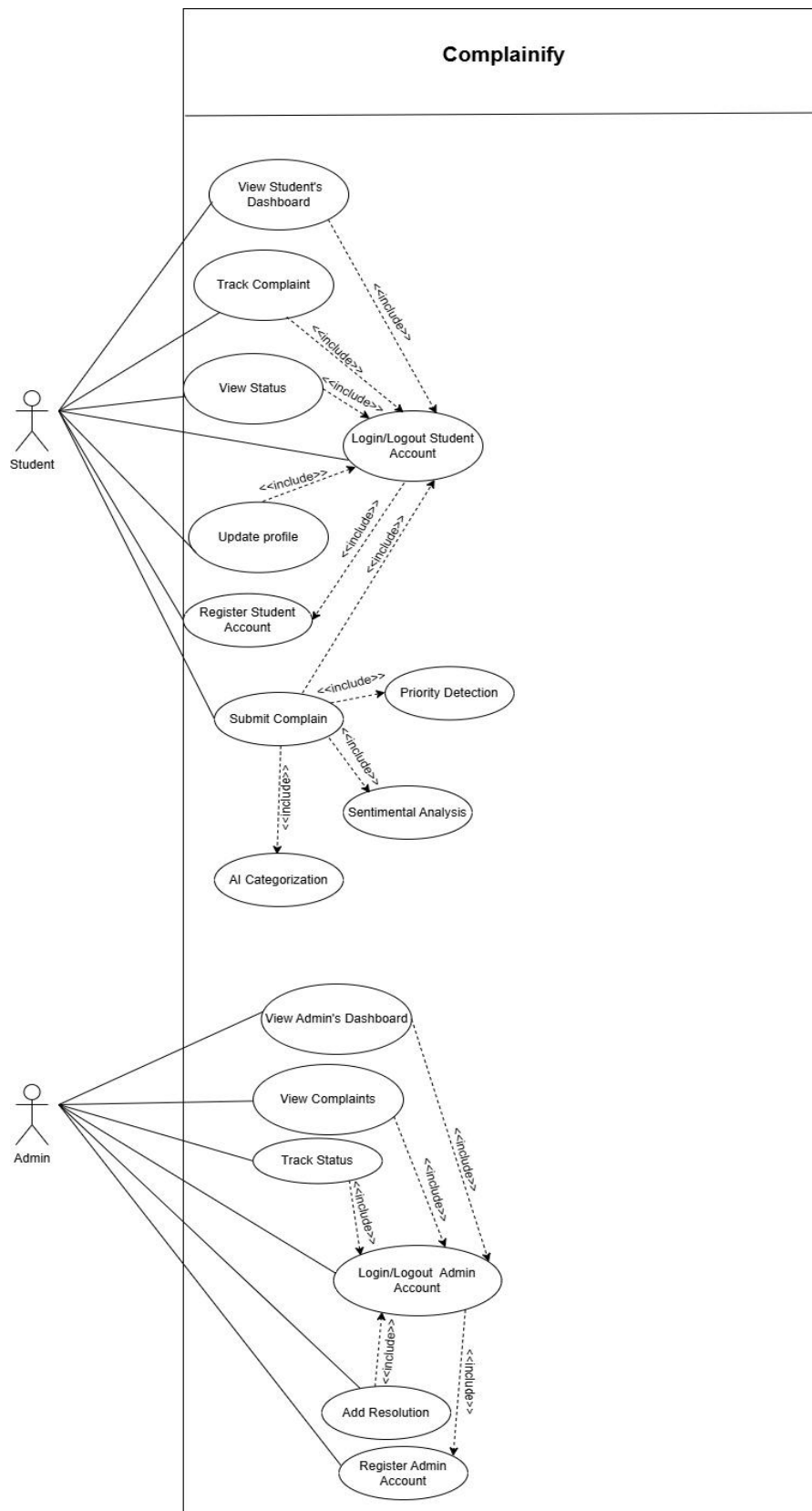


Fig 5. 4: Use Case Diagram

5.3 Database Design

5.3.1 E-R Diagram

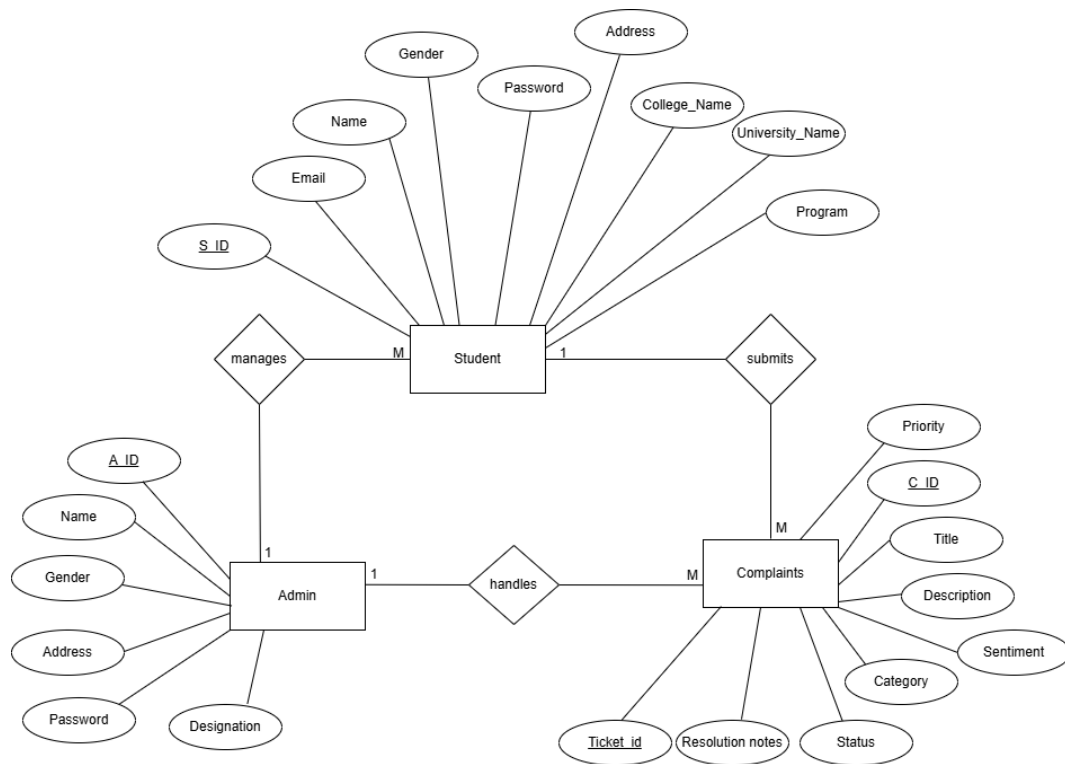


Fig 5. 5: ER Diagram

6 System Development and Implementation

6.1 Programming Platforms

1. Frontend Technologies: HTML, CSS, JavaScript

The frontend of the system is developed using HTML, CSS, and JavaScript. HTML is used to structure the web pages, CSS provides responsive styling and layout, while JavaScript adds interactivity, form validation, and dynamic content updates to enhance the user experience.

2. Backend Language: Python (Flask Framework)

The backend is developed using Python with the Flask framework. It manages user authentication, complaint processing, AI integration, database operations, email notifications, and communication between the frontend and the MySQL database.

3. Development Environment: Visual Studio Code

Visual Studio Code is used as the primary development environment for writing, debugging, and testing the application. It provides extensions, debugging tools, and integrated terminal support for efficient development.

4. Database System: MySQL

MySQL is used to store student accounts, administrator information, complaint records, AI prediction results, complaint status, and uploaded files. It provides reliable data storage and maintains relationships between database tables.

5. Additional Libraries and APIs

The system uses Flask-Mail for email notifications, Pandas and NumPy for data processing, Web Speech API is used for voice input, PyMySQL for MySQL database connectivity, and Matplotlib for generating analytical charts and reports and Web Speech API is used for voice input for the complaint description.

6.2 Hardware and Software Requirements

6.2.1 Hardware Requirements

Minimum Requirements

- Processor: Dual-Core CPU (Intel i3 or equivalent)
- RAM: 4 GB
- Storage: 500 MB free space
- Display: 1366 × 768 resolution

- Keyboard and Mouse required

Recommended Requirements

- Processor: Intel Core i5 or higher
- RAM: 8 GB
- Storage: 1 GB free space
- Display: Full HD (1920 × 1080)

6.2.2 Software Requirements

- Operating System: Windows, Linux, or macOS
- Web Browser: Google Chrome or any modern browser
- Code Editor: Visual Studio Code
- Python Environment: Anaconda
- Backend Framework: Flask
- Database: MySQL
- Programming Language: Python
- Frontend Technologies: HTML, CSS, JavaScript

7 Testing and Debugging

7.1 Testing Approach

Complainify was tested using a multi-level testing strategy. The testing covered unit testing, integration testing, and user acceptance testing. Unit tests checked individual functions such as form validation, login logic, and the Naive Bayes classification code in isolation. Integration tests confirmed that the frontend pages correctly send data to the Flask backend and that the data is saved properly in the MySQL database. User acceptance testing was done by simulating real student and admin usage scenarios to confirm that the system works correctly from the end-user point of view.

7.2 Testing Tools

Table 7.1: Testing Tools

S.N.	Tool	Specification / Purpose
1	Hardware	Laptop with 8 GB RAM, Intel Core i5 processor used to run Flask server, MySQL, and browser at the same time
2	Web Browser	Google Chrome (latest) and Mozilla Firefox used to open and test all frontend pages
3	Flask Development Server	Local Python Flask server used to run the backend application during testing
4	MySQL (XAMPP)	Relational database. Tables were checked using phpMyAdmin to confirm correct data storage
5	Postman	API testing tool used to test email notification delivery with real Gmail addresses and to verify Flask API endpoints.
6	VS Code	Code editor used for debugging Python code and checking console error messages

7.3 Test Cases

The following tables document test cases organized by module. Each test case records the test ID, description, input conditions, expected output, actual output observed

during testing, and the final status (Pass or Fail). Where initial failures were encountered during development cycles, they have been resolved and confirmed as Pass in the final build. The test IDs follow the ModuleNumber.TestNumber for easy cross-referencing.

7.3.1 Login Module

Table 7.2: Login Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-L01	Login with correct credentials	Valid student email and password	Redirect to Student Dashboard	Student Dashboard opens	Pass
TC-L02	Login with wrong password	Correct email + wrong password	Error: Invalid email or password	Error message shown	Pass
TC-L03	Login with empty fields	Leave email and password blank	Validation message shown	Required field messages shown	Pass
TC-L04	Admin login	Valid admin email and password	Redirect to Admin Dashboard	Admin Dashboard opens	Pass
TC-L05	Unregistered email	Email not in database	Error: Invalid email or password	Error message shown	Pass

7.3.2 Sign Up Module

Table 7.3: Sign Up Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-S01	Register with valid data	Name, email, password filled correctly	Account created, redirect to login	Account created successfully	Pass
TC-S02	Register with existing email	Email already in database	Error: Email already registered	Error message shown	Pass
TC-S03	Password mismatch	Password and Confirm Password different	Error: Passwords do not match	Error message shown	Pass
TC-S04	Weak password	Password less than 6 characters	Validation error for password	Validation message shown	Pass
TC-S05	Empty required fields	Leave name or email blank	Required field messages	Validation messages shown	Pass

7.3.3 Forgot Password / Reset Password Module

Table 7.4: Forgot Password/Reset Password Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-P01	Reset request with valid email	Registered email + Send Reset Link	Success message + email sent	Email received with reset link	Pass
TC-P02	Reset request with unknown email	Email not in database	Safe error message shown	Email not found message shown	Pass
TC-P03	Reset with valid token	Open reset link and set new password	Password updated successfully	Password changed successfully	Pass
TC-P04	Reset with invalid/expired token (First test)	Use old or wrong reset link	Error: Invalid or expired link	Blank page shown	Fail
TC-P04	Reset with invalid/expired token (After fix & retest)	Use old or wrong reset link	Error: Invalid or expired link	Error message shown correctly	Pass

7.3.4 Complaint Submission Module

Table 7.5: Complaint Submission Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-C01	Submit complete complaint	Title, description, category filled	Complaint saved + Ticket ID generated	Ticket ID shown, success message	Pass
TC-C02	Submit with file attachment	Upload image or PDF file	File uploaded and linked	File saved correctly	Pass
TC-C03	Submit with empty fields	Leave title or description blank	Validation error shown	Required field messages shown	Pass
TC-C04	Submit anonymous complaint (First test)	Tick anonymous option and submit	Student identity hidden from admin	Student name still visible in admin panel	Fail
TC-C04	Submit anonymous complaint (After fix & retest)	Tick anonymous option and submit	Student identity hidden from admin	Student identity hidden correctly	Pass
TC-C05	Submit long description	Description with 500+ words	System accepts and saves	Complaint saved successfully	Pass
TC-C06	Voice Complaint Input	Voice Complaint	Voice complaint converted into text.	Voice was converted into text successfully.	Pass

7.3.5 AI Classification Module (Naive Bayes)

Table 7.6: AI Classification Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-A01	Classify academic complaint	Text about exam or class issue	Category = Academics	Predicted as Academics	Pass
TC-A02	Classify hostel complaint	Text about room, water or mess	Category = Hostels	Predicted as Hostels	Pass
TC-A03	Classify fees complaint	Text about payment or scholarship	Category = Fees / Finance	Predicted as Fees / Finance	Pass
TC-A04	Classify IT complaint	Text about computer or internet	Category = IT Support	Predicted as IT Support	Pass
TC-A05	Priority assignment	Serious / urgent complaint text	Priority = High or Medium	Priority set correctly	Pass

7.3.6 Sentiment Analysis Module

Table 7.7: Sentiment Analysis Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-V01	Detect negative sentiment	Complaint with angry words	Sentiment = Negative	Marked as Negative	Pass

TC-V02	Detect neutral sentiment	Simple factual complaint	Sentiment = Neutral	Marked as Neutral	Pass
TC-V03	Detect positive sentiment	Polite or positive text	Sentiment = Positive	Marked as Positive	Pass

7.3.7 Complaint Status Tracking Module

Table 7.8: Complaint Status Tracking Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-T01	Student views own complaints	Login as student → My Complaints	List with Ticket ID and status	Complaints listed correctly	Pass
TC-T02	Admin sets status to In Progress	Admin updates status	Status changes, student can see it	Status updated correctly	Pass
TC-T03	Admin marks as Resolved	Admin sets status = Resolved	Status becomes Resolved	Status updated, student sees it	Pass
TC-T04	Student cannot see others	Student A tries to open Student B complaint	Access denied / not visible	Complaint not visible	Pass

7.3.8 Admin Dashboard and Filtering Module

Table 7.9: Admin Dashboard and Filtering Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-D01	View dashboard statistics	Login as admin → Dashboard	Total count, charts visible	Statistics and charts shown	Pass
TC-D02	Filter by category	Select category = Hostels	Only Hostels complaints shown	Filter worked correctly	Pass
TC-D03	Filter by priority	Select priority = High	Only High priority listed	Filter worked correctly	Pass
TC-D04	Search by Ticket ID	Enter valid Ticket ID	Matching complaint displayed	Correct complaint shown	Pass
TC-D05	Filter by status	Select status = Resolved	Only resolved complaints shown	Filter worked correctly	Pass

7.3.9 Email Notification Module

Table 7.10: Email Notification Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-E01	Email after submission	Student submits complaint	Confirmation email with Ticket ID	Email received correctly	Pass

TC-E02	Email on status update (First test)	Admin changes status	Student receives update email	No email was sent	Fail
TC-E02	Email on status update (After fix & retest)	Admin changes status	Student receives update email	Email sent successfully	Pass
TC-E03	Email on resolution	Admin marks as Resolved	Resolution email sent	Email received correctly	Pass

7.3.10 Report Generation Module

Table 7.11: Report Generation Module

TC ID	Description	Input	Expected Output	Actual Output	Status
TC-R01	Generate Excel report	Admin clicks Export Excel	Excel file downloads	Excel file downloaded	Pass
TC-R02	Generate PDF report (First test)	Admin clicks Export PDF	PDF file downloads	Error occurred, no file downloaded	Fail
TC-R02	Generate PDF report (After fix & retest)	Admin clicks Export PDF	PDF file downloads	PDF downloaded correctly	Pass

8 Conclusion

Complainify is a web-based AI-powered student complaint management system that successfully achieves its primary objectives, such as secure multi-role login and registration, complaint submission with file upload support, anonymous complaint option, automatic category prediction using Multinomial Naive Bayes priority assignment (High, Medium, Low), sentiment analysis (implemented from scratch), complaint status tracking with Ticket ID, admin dashboard with filtering and search, email notifications for status updates, voice based complain input (English Only) and report generation in PDF and Excel formats. The system is monitored by the administrator and is developed using the Iterative Waterfall model as an SDLC approach. During testing and debugging, a couple of features were identified as failed, but they were later fixed, and overall, the testing and debugging showed positive results.

Although it achieves its primary objectives, there are some limitations, such as a lack of multi-language support, no integration with existing university management systems, and the absence of advanced AI-based recommendation or predictive resolution features. Additionally, the system currently relies only on email notifications and does not support real-time mobile push notifications, which can be improved as a future enhancement. Other future improvements may include multi-language support (especially Nepali), AI-based chatbot for students, mobile application, and integration with the college ERP system. During development, we also faced several challenges, such as implementing Naive Bayes classification, Sentimental and Priority Analysis from scratch without using Scikit-learn, training the model with limited labeled data, integrating email services correctly, and occasional XAMPP port conflicts which sometimes caused temporary database connection issues. Overall, we enjoyed working on this project and gained practical knowledge of implementing core web technologies along with Artificial Intelligence and Data Mining concepts.

References

- Aniket, D., Tanmay, G., Vaishnavi, G., Rutuja, G., & Narode, P. (2025). Smart complaint management system with AI-powered prioritization and escalation. *International Journal of Creative Research Thoughts (IJCRT)*, 13(10), f295–f300. Retrieved June 14, 2026, from <https://ijcrt.org/papers/IJCRT2510624.pdf>
- Javed, R., Wasi, S., Mughal, M. H., & Bhutto, Z. A. (2025). Machine learning-based smart students' complaint resolution system. *International Journal of Artificial Intelligence & Mathematical Sciences*, 4(1), 36–55. Retrieved June 14, 2026, from <https://doi.org/10.58921/ijaims.v4i1.135>
- Prakash, J., Swathiramya, R., Balambigai, G., Menaha, R., & Abhirami, J. S. (2024). AI-driven real-time feedback system for enhanced student support: Leveraging sentiment analysis and machine learning algorithms. *International Journal of Computational and Experimental Science and Engineering*, 10(4). Retrieved June 14, 2026, from <https://doi.org/10.22399/ijcesen.780>